CS2310 Take-home 5: Control unit and core instruction set

Group 38 | Roll number: **CS25B035** | Logisim-Evolution 4.1.0 | G38_th5_control.circ, G38_th5_microcode_gen.py, G38_th5_control_rom.hex, G38_th5_tests/

Summary. The manual switch panel of our TH4 datapath is driven by a microprogrammed control unit: a 4-bit T-state register and a 2048 x 32 control ROM addressed by {Z, N, OPCODE[4:0], TSTATE[3:0]}, with Z and N taken from the latched outputs of status_reg. The ROM image is produced by G38_th5_microcode_gen.py, which passes all its consistency checks (7/7). All fifteen core instructions are implemented in microcode and every test program, including the boundary cases, ends with registers and PC identical to TARA Studio (13/13). **Condition A** (four consecutive fetches under ROM control) and **Condition B** (programmatically generated ROM) are both met. Core pool claimed: 43/43.

1. Interface table

New or changed in TH5 (all ports, exactly as labelled in the file):

Subcircuit	Inputs	Outputs	Status
tara_cpu (main)	RST; internal Clock component CLK	(none; values are observed on the subcircuit pins)	new
control_unit	CLK, RST, Z, N, opcode[5]	cw[32], T_STATE[4], HALTED, HALT	new
tara_datapath	CLK, RST, PC_OUT, IR_OUT, MDR_OUT, IMM_OUT, RF_OUT, ALU_OUT, RAM_OUT_LD, MAR_LOAD, MDR_LOAD, IR_LOAD, ACC_load, RF_WRITE, PC_EN, PC_LOAD, mem_read, mem_write, ALU_select[5], ALU_A_SEL, ALU_B_SEL, status_we, IMM_SEL[3], RA_SEL[2]	opcode[5], Z, N, PC[16], IR[16]	changed: ALU operand muxes, RA index mux, imm_unit, ACC, status_reg, outputs to control unit
imm_unit	imm8[8], off5[5], rel8[8], rel11[11], QA[16], IMM_SEL[3]	IMM[16]	new
ram_word	addr[16], din[16], mem_read, mem_write, clk	dout[16]	changed: asynchronous read
regfile8x16	ra[3], rb[3], rd[3], din[16], we, clk, rst, ra_enb, rb_enb	qa[16], qb[16]	fixed (R7 to port B)

Reused unchanged from TH1-TH4:

Subcircuit	Ports	Subcircuit	Ports
ALU	A[16], B[16], OPCODE[5], ALU_OUTPUT_ENABLE → BUS_OUT[16], ZERO, NEGATIVE, CARRY, OVERFLOW	tara_decoder	IR[16] → op[5], format[3], RD, RA, RB[3], imm8[8], off5[5], rel8[8], rel11[11]
status_reg	Z_in, N_in, C_in, V_in, status_we, clk, RST → Z, N, C, V	PC_MAR_MDR_IR	CLK, RST, MAR_D/EN, MDR_D/EN, IR_D/EN, PC_D, PC_LOAD, PC_EN → PC_Q, MAR_Q, MDR_Q, IR_Q
register16	d[16], clk, en, rst → q[16] (used as ACC)	dff	D, CLK, EN, RST → Q
adders, addsub16	half/full adder, adder4, adder16, addsub16 (SUB, OVF)	shifters, comparators	SHL16/SHR16 (A, SH[5] → Y), shiftL/Rby1..16, cmp1/cmp4/cmp16
ALU internals	mux_tree, conditional_logic, Tri_state_buffer, op_to_format		

2. Control word (one field per datapath input pin; 29 of 32 bits used)

Bits	Field	Meaning when 1 / value	Bits	Field	Meaning when 1 / value
0	PC_OUT	PC → bus	12	PC_EN	PC update (with PC_LOAD=0: PC+2)
1	IR_OUT	IR → bus	13	PC_LOAD	with PC_EN: PC ← bus (TH6)
2	MDR_OUT	MDR → bus	14	mem_read	memory output enable
3	IMM_OUT	imm_unit → bus	15	mem_write	M[MAR] ← MDR (TH6)
4	RF_OUT	register port A (qa) → bus	16-20	ALU_select	ALU opcode (9 ADD ... 19 SLT)
5	ALU_OUT	ALU result → bus	21	ALU_A_SEL	ALU A: 0 qa, 1 PC
6	RAM_OUT_LD	memory → bus	22	ALU_B_SEL	ALU B: 0 qb, 1 IMM
7	MAR_LOAD	MAR ← bus	23	status_we	latch Z N C V
8	MDR_LOAD	MDR ← bus	24-26	IMM_SEL	0 zext8, 1 sext8, 2 sext5, 3 rel8x2, 4 rel11x2, 5 const 2, 6 {imm8,QA[7:0]}, 7 zero
9	IR_LOAD	IR ← bus	27-28	RA_SEL	port A reads 0 IR[7:5], 1 IR[10:8], 2 R6, 3 R7
10	ACC_load	ACC ← bus	29	STEP_RESET	next edge: T-state ← 0
11	RF_WRITE	R[IR[10:8]] ← bus	30	HALT	set HALTED; control word then forced to 0

3. Design decisions

Decision	Justification
Control word is the TH4 switch panel.	Each field is one input pin of tara_datapath with the same name, so the inventory from TH4 became the control word unchanged and the split in tara_cpu is a single 32-bit splitter. The seven bus sources keep separate enables; the course recommends an encoded field, and we compensate in the generator: check 1 rejects any word with two sources and check 4 rejects any load from an undriven bus.
ROM addressed by {Z, N, OPCODE, TSTATE}.	2048 x 32 words, Z and N from status_reg's latched outputs. In TH5 every word is identical for all four (Z, N) combinations (check 5); the duplication keeps the TH6 branches simple.
T-state counter and HALT.	A 4-bit register with a +1 adder; a 2:1 mux selects 0 when STEP_RESET (cw bit 29) is set. HALT (bit 30) sets a D flip-flop that holds itself through an OR gate; while HALTED=1 a 32-bit mux forces the control word to 0, so nothing can change until RST. PC has already advanced past HLT, as in TARA Studio.
Fetch in three states.	T0 0x00000081 (PC_OUT, MAR_LOAD); T1 0x00005140 (mem_read, RAM_OUT_LD, MDR_LOAD, PC_EN); T2 0x00000204 (MDR_OUT, IR_LOAD). The RAM reads asynchronously, so MDR captures M[MAR] at the end of T1 (see fault 2).
Every core instruction executes in one state (T3).	ALU operands come straight from the register-file ports (A = qa or PC, B = qb or IMM) and only the result uses the bus, so a register-register operation needs no temporary register. Each instruction takes 4 clock cycles.
RA index mux (RA_SEL).	LIH, ADDI, SHL and SHR operate on Rd, so read port A can be switched from IR[7:5] to IR[10:8]. Inputs 2 and 3 are the constants 6 and 7, already wired for RET and PUSH/POP in TH6.

Decision	Justification
LIH as read-modify-write in a single state.	Port A reads Rd, imm_unit forms {imm8, QA[7:0]} (IMM_SEL=6) and the result is written back to Rd, so the low byte is kept. Tested by building ABCD with LIL+LIH and by LIH over a non-zero low byte (t03).
ADDI sign-extends; LIL zero-extends.	IMM_SEL=1 (sext imm8) with RA_SEL=1 so Rd is both source and destination; LIL uses IMM_SEL=0. Tested with -1, -128, +127 and 7FFF+1 (t07).
SHL / SHR take the amount from the immediate.	ALU_B_SEL=1 selects imm_unit (zext imm8); the ALU uses B[4:0], so amounts 16-31 give 0 and 0x21 shifts by 1, exactly as TARA Studio (t09).
status_we on every ALU instruction; ACC loaded with every ALU result.	No core instruction reads Z/N, but latching them keeps the last condition visible on the probes and the field is ready for BZ/BN. ACC is an observation copy of the last ALU result.
Unimplemented opcodes HALT; unused ROM words step-reset.	MUL (12), opcodes 27-31 and the TH6 instructions get HALT at T3, so they cannot modify state (check 6, test t21). Every other unused address holds STEP_RESET, so a stray T-state only wastes one cycle.
Memory: big-endian, two byte banks.	ram_word keeps two 8-bit RAMs addressed by addr[15:1]; the top RAM holds the low byte and the bottom RAM the high byte, so M[even] is the high byte of the word, as in TARA Studio. A program is loaded as two images (_LOW / _HIGH).

4. Microprograms (T0-T2: common fetch; all end with STEP_RESET)

Instr.	opcode	T3 word	Signals	Effect
NOP	00000	0x20000000	STEP_RESET	-
HLT	00001	0x60000000	HALT, STEP_RESET	stop
MOV	00010	0x20000810	RF_OUT, RF_WRITE	Rd ← Rs
LIL	00011	0x20000808	IMM_SEL=0, IMM_OUT, RF_WRITE	Rd ← zext(imm8)
LIH	00100	0x2E000808	RA_SEL=1, IMM_SEL=6, IMM_OUT, RF_WRITE	Rd ← {imm8, Rd[7:0]}
ADD SUB AND OR XOR SLT	9 10 13 14 15 19	0x20890C20 ...	ALU_select, ALU_OUT, RF_WRITE, ACC_load, status_we	Rd ← Ra op Rb
NOT	10000	0x20900C20	ALU_select=16, ALU_OUT, RF_WRITE, ACC_load, status_we	Rd ← ~Rs
ADDI	01011	0x29C90C20	RA_SEL=1, ALU_B_SEL, IMM_SEL=1, ALU_select=9, ALU_OUT, RF_WRITE ...	Rd ← Rd + sext(imm8)
SHL / SHR	17 / 18	0x28D10C20 / 0x28D20C20	RA_SEL=1, ALU_B_SEL, IMM_SEL=0, ALU_OUT, RF_WRITE ...	Rd ← Rd <</>> imm8[4:0]
MUL, 27-31, TH6 ops		0x60000000	HALT, STEP_RESET	policy: halt

Generator log (G38_th5_microcode_log.txt), Condition B:

```
TARA microcode generator, TH5 — 15 instructions: NOP HLT MOV LIL LIH ADD SUB ADDI AND OR XOR NOT SHL SHR SLT
ROM 2048 x 32 bits

[PASS] 1. no control word selects more than one bus source
[PASS] 2. every instruction eventually asserts STEP_RESET or HALT
[PASS] 3. mem_read and mem_write are never asserted together
[PASS] 4. write enables are never asserted without valid source data
[PASS] 5. conditional-branch words differ only in the relevant condition signal
[PASS] 6. no undefined opcode can modify architectural state (they HALT)
[PASS] 7. T0-T2 hold the common fetch sequence for every opcode

7/7 checks passed
```

5. Test evidence

Method. Every program in G38_th5_tests/programs is assembled with the TARA Studio assembler, loaded into ram_word as two byte images, and run on our circuit until HALTED=1; the same program is run on TARA Studio's reference CPU model. All screenshots below are of our own circuit in Logisim after HLT. Full per-instruction reference logs are in G38_th5_tests/reference_traces.

Program	Instructions tested	R0 ... R7 after HLT (ours = TARA Studio)	PC	
t01	fetch x4, NOP, HLT	0000 0000 0000 0000 0000 0000 0000 0000	000A	PASS
t02	LIL	00FF 0080 0000 0000 0000 0000 0000 002A	000A	PASS
t03	LIH (low byte kept)	ABCD FF34 0500 00FF 0000 0000 0000 0000	0012	PASS
t04	MOV	0000 1234 1234 0000 0000 0000 0000 1234	000E	PASS
t05	ADD, 7FFF+1, FFFF+1	0000 7FFF 0002 8000 FFFF 0000 0002 0000	0014	PASS
t06	SUB, 3-5, 8000-1	0000 0003 0005 FFFE 0002 8000 0001 7FFF	0014	PASS
t07	ADDI -1, +127, -128, 7FFF+1	0000 0008 FFFF 8000 007F 0000 0000 0000	0016	PASS
t08	AND OR XOR NOT	FFFF 0FF0 A53C 0530 AFFC AACC 0000 F00F	0016	PASS
t09	SHL/SHR by 0,1,3,15,16,33	0000 0028 8000 0000 4000 0001 00FF 7FFF	0024	PASS
t10	SLT signed, equal, 8000 vs 7FFF	0001 FFFF 0001 0001 0000 0000 0000 7FFF	001A	PASS
t20	every register on ports A and B	FEE1 FFEF FFEF FFEF FFEF FFEF FFEF 010E	0026	PASS
t21	MUL → HALT, nothing changed	0000 0003 0000 0000 0000 0000 0000 0000	0004	PASS*
p1	register addition (regression)	0000 1234 0FED 2221 0000 0000 0000 0000	000C	PASS

* t21 is checked against our documented policy: TARA Studio executes MUL, our processor halts at it with no state change.

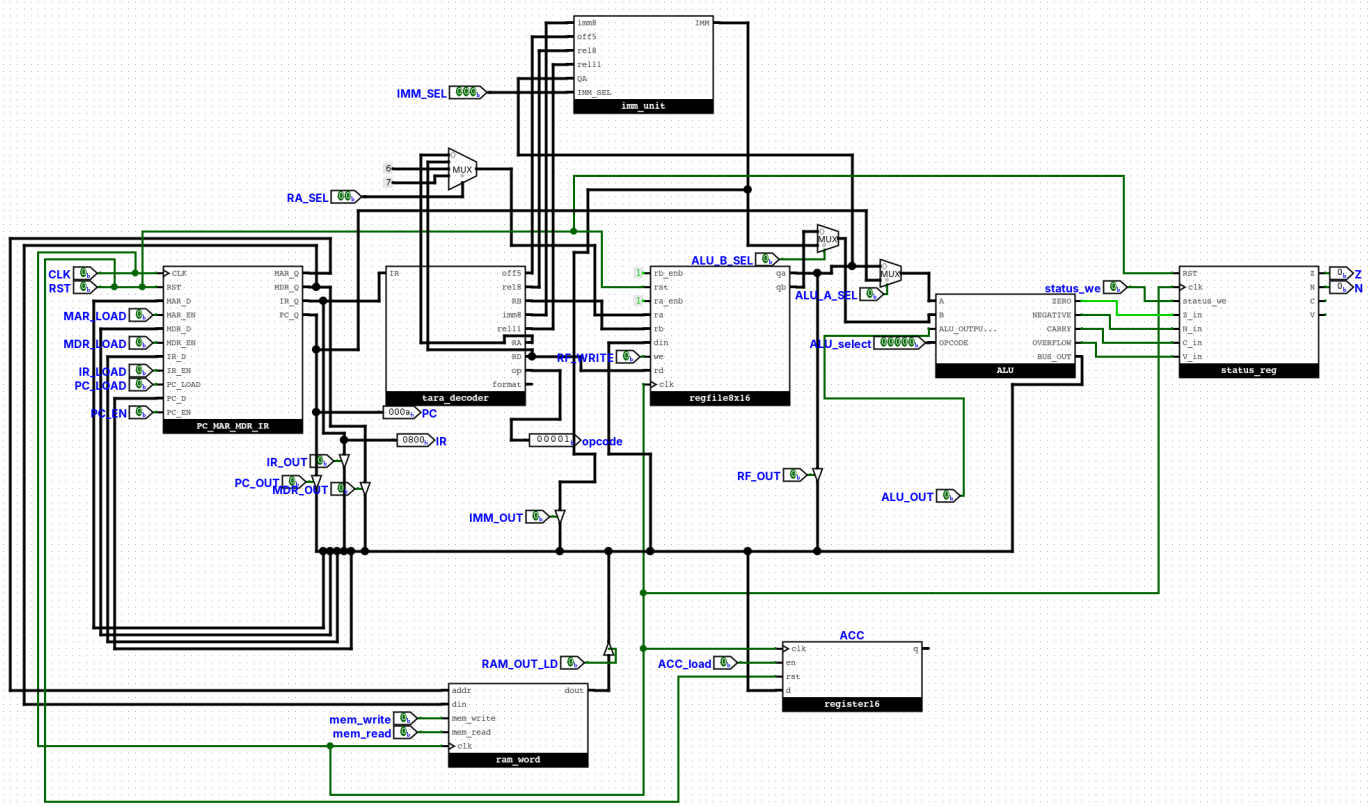


Figure 1. Condition A: tara_datapath after t01 (NOP x4, HLT). Four fetches completed under ROM control: PC = 000A, IR = 0800 (HLT), opcode = 00001. Control inputs are all 0 because the halted control unit forces cw = 0.

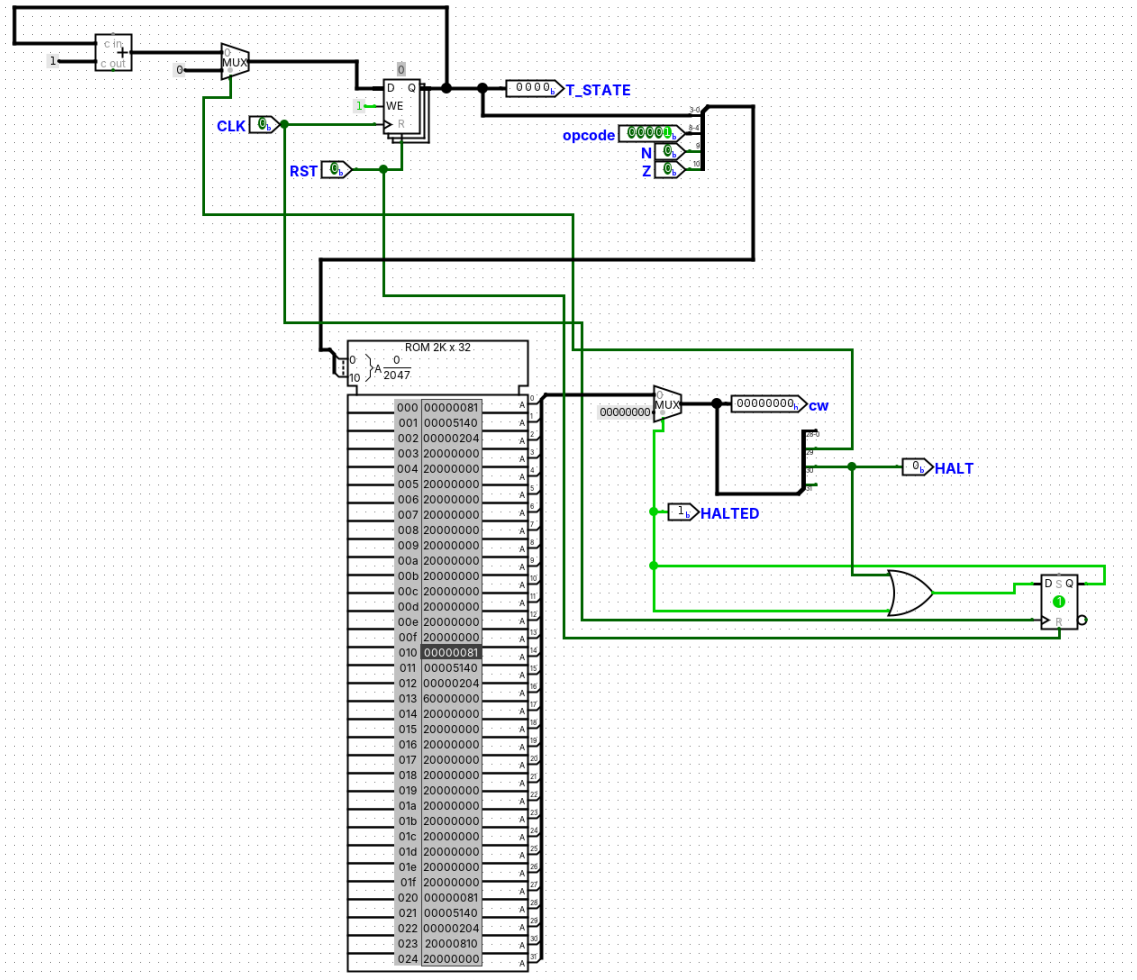


Figure 2. control_unit after t01: HALTED = 1, T_STATE = 0, cw masked to 0. The ROM shows the generated words: 081, 5140, 204 at T0-T2 of every opcode, 6000000 at HLT T3 (address 0x013).



Figure 3. Register file (R0 top to R7 bottom) of our circuit after HLT for ten test programs; every column equals the expected values in the table above (e.g. t03 R0 = abcd, t05 R3 = 8000, t09 R4 = 4000, p1 R3 = 2221).

6. Faults encountered and how they were found

Fault	Symptom, diagnosis and fix
Register file could not read R7 on port B.	SUB and SLT returned E (conflict) in bit 0 whenever R7 was the second operand, while R2/R3 worked. Minimal programs isolated it to port B with rb=7: the wire from R7 to its port-B tri-state buffer in regfile8x16 ended at (830,1020), 10 units short of the buffer input at (830,1030), so qb floated. Fixed by completing the wire; test t20 now reads every register through both ports.
Every fetch returned the previous instruction.	Registers were right in straight-line code but PC finished 2 too high. A cycle trace showed MDR capturing the word of the old MAR: Logisim's RAM reads on the clock edge by default, the same edge at which MDR samples. Setting Asynchronous read = Yes on both byte RAMs made the output follow MAR within the state.
TH4 ALU operands passed through controlled inverters.	ALU A was NOT(bus) gated by ACC_LOAD and B was NOT(qa) gated by a format decoder, so operands were inverted or floating, and A formed a loop through the bus. Replaced by A = mux(qa, PC) and B = mux(qb, IMM) with ALU_A_SEL and ALU_B_SEL; ACC became a real register16. Also fixed: qa and qb shorted by a wire at (1200,220)-(1200,240), ra_enb/rb_enb floating, ALU OPCODE unconnected.
T-state counter stuck at T1.	Headless run of control_unit with opcode HLT: cw went 0x81, 0x5140, then stayed 0x5140. The +1 adder's input came from a stray input pin 'Q' instead of the T_STATE register output, so it always computed 0+1. Fixed by wiring the register output back to the adder; HLT then halts after T3 and ADD returns to T0.
RA mux select unconnected.	After adding the RA index mux, a netlist check showed its 2-bit select connected to nothing, which would leave port A undefined for every instruction. Added the RA_SEL pin.
Every program using LIH failed after integration.	LIL-only and register tests passed, but any constant built with LIL+LIH ended as xxxx. Test vectors on our imm_unit gave {AB, UU} for IMM_SEL=6 and UUU0 for IMM_SEL=4: the QA and rel11 input pins sat 10 units away from their tunnels. Wired both; imm_unit then passed 210/210 vectors and all 13 programs passed.
opcode wire ran along the datapath's output edge.	In tara_cpu the opcode wire went down x=1730, the column of the datapath's output ports; any further output would have been shorted onto it. Rerouted away from the box (preventive, found while reviewing the netlist).